

Threat Hunting in AWS Using Network Firewall, DNS Firewall, and CloudWatch

Technical Blog Report - Centralized Inspection, DNS Exfiltration Detection, and Incident Response

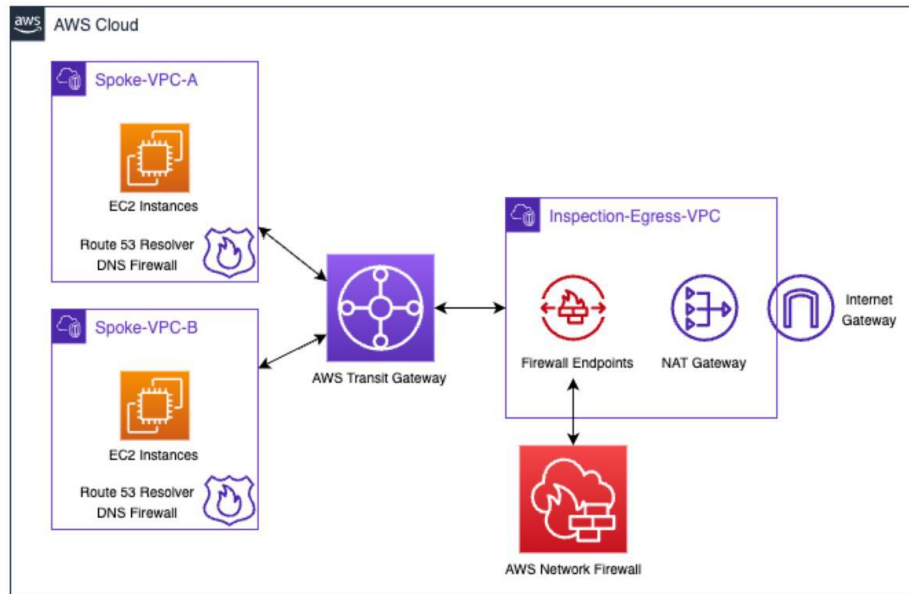


Figure 1 - Hub-and-spoke AWS architecture with centralized egress inspection through AWS Network Firewall.

Scope: AWS Network Firewall, Route 53 Resolver DNS Firewall, CloudWatch Log Insights, Contributor Insights, and EC2 containment.

1. Executive Summary

Modern AWS environments often span multiple VPCs, private subnets, NAT gateways, and service-specific routing paths. That design is scalable, but it can also reduce network visibility if outbound traffic is not inspected centrally. This project implements a threat hunting architecture that forces spoke VPC traffic through an inspection VPC, applies AWS Network Firewall controls, blocks suspicious DNS resolution with Route 53 Resolver DNS Firewall, and uses CloudWatch to identify compromised EC2 instances.

The investigation focuses on three post-compromise behaviors: network reconnaissance, protocol abuse on TCP/443, and DNS-based data exfiltration. The result is a layered detection and response workflow: inspect traffic, generate telemetry, hunt in logs, identify affected workloads, and quarantine suspicious EC2 instances.

Area	Implementation	Security outcome
Network architecture	Hub-and-spoke routing through an Inspection/Egress VPC	Centralized visibility and egress enforcement
Egress filtering	AWS Network Firewall stateful rule groups	Blocks suspicious domains and detects anomalous protocols
DNS security	Route 53 Resolver DNS Firewall	Blocks malicious DNS queries and exposes exfiltration attempts
Threat hunting	CloudWatch Log Insights and Contributor Insights	Attributes suspicious behavior to EC2 instances
Response	Quarantine security group	Contains compromised workloads while preserving evidence

2. Security Problem

The starting problem is not simply that workloads can reach the internet. The problem is that compromised workloads may use permitted outbound paths to perform reconnaissance, connect to suspicious infrastructure, tunnel unexpected protocols, or exfiltrate data using DNS. Traditional security groups and NACLs are insufficient for this type of investigation because they do not provide deep protocol inspection or high-fidelity DNS telemetry.

The security questions for this project were:

- Can outbound traffic from multiple spoke VPCs be forced through a central inspection layer?
- Can suspicious HTTP/HTTPS destinations be blocked at the network layer?
- Can protocol anomalies such as non-TLS traffic over port 443 be detected?
- Can DNS queries be blocked and analyzed for exfiltration indicators?
- Can the source EC2 instances be identified and contained?

3. Architecture Overview

The environment uses a hub-and-spoke network model. Two workload VPCs contain EC2 instances. A separate Inspection/Egress VPC contains AWS Network Firewall endpoints, a NAT Gateway, and an Internet Gateway. AWS Transit Gateway connects the VPCs and routes outbound traffic from the spoke VPCs into the inspection layer before traffic exits AWS.

```
EC2 workload
-> Security Group / NACL
-> AWS Transit Gateway
-> Firewall Endpoint
-> AWS Network Firewall
-> NAT Gateway
```

```
-> Internet Gateway
-> Internet
```

This design creates a choke point for outbound traffic. Instead of applying isolated controls per VPC, the architecture centralizes inspection, logging, and enforcement in the Inspection/Egress VPC.

4. Attacker Behavior Simulated

The investigation simulates attacker behavior that may occur after a workload compromise. The scenarios are intentionally network-focused because the goal is to validate whether cloud-native network telemetry can expose suspicious activity without relying on host-based agents.

Scenario	Observed behavior	Likely objective	MITRE ATT&CK
Network reconnaissance	Repeated ICMP traffic from a single EC2 source	Host discovery and route mapping	T1046 - Network Service Discovery
Protocol tunneling	FTP-like traffic observed over TCP/443 instead of TLS	Bypass controls that expect HTTPS on port 443	T1071 - Application Layer Protocol
DNS exfiltration	Large bursts of long, Base64-like subdomains	Exfiltrate credentials through DNS queries	T1048.003 - Exfiltration Over DNS

4.1 Network Reconnaissance

One instance generated repeated ICMP echo requests. ICMP is not malicious by itself, but repeated activity from a single host at regular intervals is a useful hunting signal. In this context, the behavior is consistent with host discovery or environment mapping.

4.2 Protocol Tunneling on Port 443

Another instance generated traffic on TCP port 443 where the application protocol was not TLS. This matters because port 443 is frequently allowed through egress controls under the assumption that it carries HTTPS traffic. Non-TLS traffic on this port can indicate protocol tunneling, command-and-control, or an attempt to bypass policy assumptions.

4.3 DNS Data Exfiltration

A third workload attempted to send data by embedding encoded payloads in DNS query names. Long subdomains, repeated queries to the same domain, and Base64-like strings are strong indicators of DNS tunneling or DNS-based exfiltration.

```
Example pattern:
<base64_payload>.example.xyz

Decoded payload summary:
Name, AccessKey, SecretAccessKey
```

5. Defensive Controls Implemented

The defensive design uses multiple controls that complement each other. Network Firewall handles egress inspection and Suricata-compatible detection. DNS Firewall blocks name resolution and provides query telemetry. CloudWatch turns both data sources into a hunting workflow.

5.1 AWS Network Firewall - Centralized Egress Inspection

AWS Network Firewall was deployed in the Inspection/Egress VPC. Firewall endpoints receive traffic from Transit Gateway and forward inspected traffic toward the NAT Gateway. This makes the firewall a central policy enforcement point for internet-bound traffic from the spoke VPCs.

IMPLEMENTATION SUMMARY

- Create the firewall in the Inspection/Egress VPC.
- Deploy firewall endpoints in dedicated subnets.
- Attach the firewall to InspectionFirewall-Policy.
- Route spoke VPC egress through Transit Gateway and the firewall endpoints.

5.2 Domain Block List Rule Group

A stateful domain list rule group was created to deny HTTP and HTTPS traffic to simulated low-reputation domains. This is a preventive control and a telemetry source: blocked or matched traffic can later be inspected in CloudWatch.

```
Rule group: RestrictedDomains
Type: Stateful domain list
Scope CIDR: 10.0.0.0/8
Protocols: HTTP, HTTPS
Action: Deny
```

```
Domains:
.example.xyz
.example.stream
.example.party
.example.click
.example.win
.example.download
.example.bid
.example.vip
.example.net
```

The screenshot shows the 'Configure rules' interface for an AWS Network Firewall. It details the configuration for a 'Domain list rule' with the following settings:

- Domain list rule info:** Allow or deny traffic based on the domain name list.
- Domain names:** A text area containing the list of domains: `.example.bid`, `.example.vip`, and `.example.net`. Below the text area is the instruction: 'Enter one domain name per line.'
- CIDR ranges:** A section for 'The source traffic CIDR ranges to inspect.' It has two radio buttons: 'Default' (selected) and 'Custom'. The 'Default' option is described as 'Use the CIDR range of the VPC where Network Firewall is deployed.'
- CIDR ranges:** A text area containing the CIDR range `10.0.0.0/8`. Below the text area is the instruction: 'Enter one CIDR range per line.'
- Protocols:** A section for 'The protocols to inspect.' It has two checkboxes: 'HTTP' (checked) and 'HTTPS' (checked).
- Action:** A section for 'Action to take when a request matches the domain names in this group.' It has four radio buttons: 'Allow' (selected), 'Alert', 'Deny', and 'Reject'.

At the bottom right of the configuration panel are three buttons: 'Cancel', 'Previous', and 'Next'.

Figure 2 - AWS Network Firewall stateful domain list rule group blocking suspicious HTTP/HTTPS destinations.

The rule was validated from an EC2 command host using curl. A denied domain timed out, while an unrelated domain was still reachable. This confirms that the blocklist was scoped to the specified domain patterns rather than breaking general egress connectivity.

```
curl https://www.example.net --max-time 5
```

Expected result:

```
curl: (28) Connection timed out after 5000 milliseconds
```

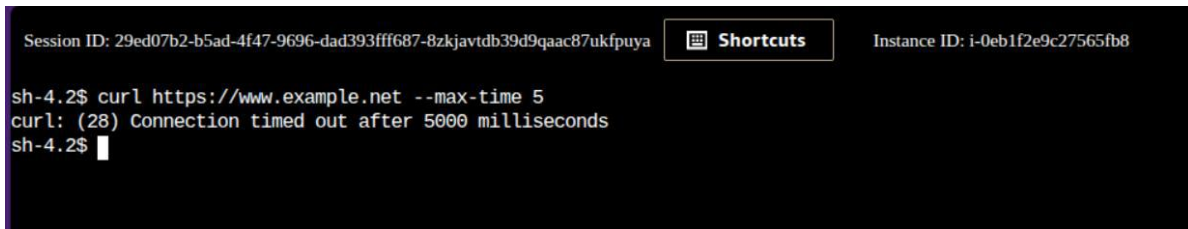


Figure 3 - Domain block validation showing a denied request to a restricted domain.

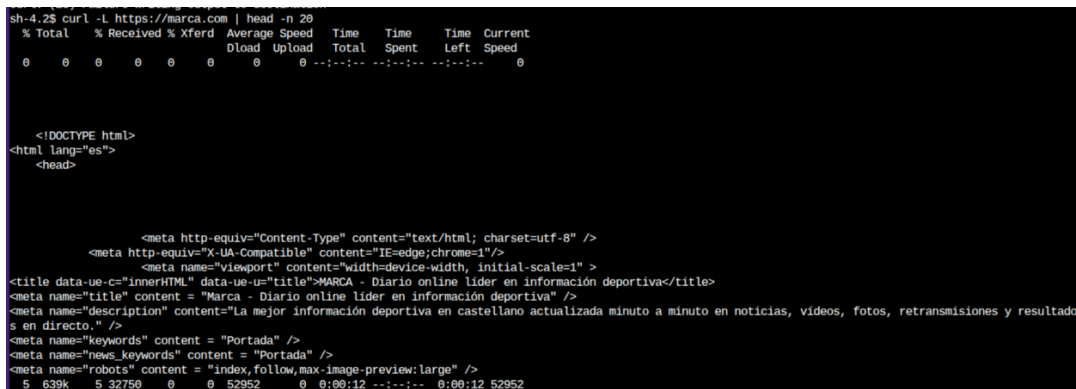


Figure 4 - Control test showing that non-restricted domains remain reachable through the firewall.

5.3 Suricata Detection for Non-TLS Traffic on Port 443

A custom Suricata-compatible rule was deployed to detect TCP sessions using port 443 where the application-layer protocol is not TLS. This rule is useful because it detects a mismatch between expected port behavior and observed protocol behavior.

```
alert tcp any any <> any 443 (msg:"SURICATA Port 443 but not TLS";
flow:to_server,established;
app-layer-protocol:!tls;
sid:2271003;
rev:1;)
```

Rule component	Purpose
alert tcp	Generate an alert for matching TCP traffic
any any <> any 443	Inspect traffic involving TCP port 443
flow:to_server,established	Evaluate established sessions directed to the server
app-layer-protocol:!tls	Trigger when the detected application protocol is not TLS
sid / rev	Signature identifier and revision metadata

VPC > Network Firewall: Firewalls > InspectionFirewall > Create and add stateful rule group

A firewall policy can be associated with multiple firewalls. Modifying a firewall policy affects all firewalls that reference it. To use rule groups that are managed for you, see [AWS Partner Network \(APN\) integrations](#).

Step 1: Choose rule group type

Step 2: Describe rule group

Step 3: Configure rules

Step 4 - optional: Configure advanced settings

Step 5 - optional: Add tags

Step 6: Review and create

Choose rule group type

Network Firewall rule groups are either stateless or stateful. Stateless rule groups evaluate packets in isolation, while stateful rule groups evaluate them in the context of their traffic flow.

Rule group type

☒ Stateful rule group
Use stateful rule groups to inspect packets within the context of the traffic flow.

☐ Stateless rule group
Use stateless rule groups to inspect individual packets on their own, without the context of the traffic flow.

Rule group format

Suricata compatible rule string

Figure 5 - Custom Suricata rule group used to detect non-TLS traffic on TCP port 443.

Stateful rule groups (3)

Actions

☐	Name	Capacity	Managed	
☐	IcmpAlert-RuleGroup	100	No	
☐	RestrictedDomains	100	No	
☐	NonTlsTrafficOn443	10	No	Not available

Create stateful rule group

Add unmanaged stateful rule groups

Add managed stateful rule groups

Disassociate from policy

Rule group details

Figure 6 - Firewall policy with attached stateful rule groups including domain filtering and Suricata detection.

5.4 AWS Managed Threat Intelligence Rule Groups

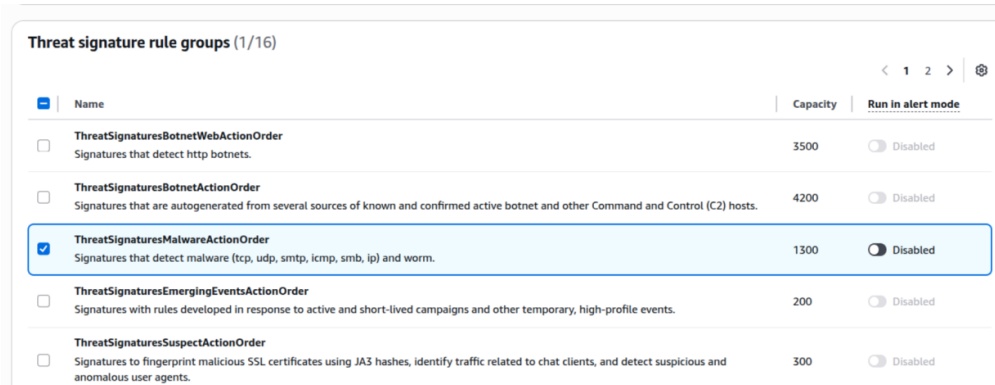
Custom rules are useful for environment-specific detections, but they cannot keep pace with the full threat landscape. AWS managed rule groups were added to extend coverage using AWS-maintained domain, IP, and malware threat intelligence.

- Domain and IP reputation rule groups
- ThreatSignaturesMalwareWebActionOrder
- Capacity monitoring to preserve room for future expansion

Domain and IP rule groups (4/4)

☒	Name	Capacity	Run in alert mode
☒	AbusedLegitBotNetCommandAndControlDomainsActionOrder Contains rules that allow you to block requests to a class of domains which are generally legitimate but are compromised and may host botnets. This can help reduce the risk of resources accessing botnets originating from these sources with poor reputation.	200	☐ Disabled
☒	MalwareDomainsActionOrder Contains rules that allow you to block requests to domains that are known for hosting malware. This can help reduce the risk of receiving malware or viruses originating from these known sources.	200	☐ Disabled
☒	BotNetCommandAndControlDomainsActionOrder Contains rules that allow you to block requests to domains that are known for hosting botnets. This can help reduce the risk of resources accessing botnets originating from these known sources	200	☐ Disabled
☒	AbusedLegitMalwareDomainsActionOrder Contains rules that allow you to block requests to a class of domains which are generally legitimate but are compromised and may host malware. This can help reduce the risk of receiving malware or viruses originating from these sources with poor reputation.	200	☐ Disabled

Figure 7 - AWS managed domain and IP reputation rule groups enabled in the firewall policy.



The screenshot shows the 'Threat signature rule groups' page in the AWS console. It displays a list of five rule groups, all of which are currently disabled. The 'ThreatSignaturesMalwareActionOrder' rule group is highlighted with a blue selection bar. This rule group is designed to detect malware signatures across various protocols including TCP, UDP, SMTP, ICMP, SMB, and IP, and is known for detecting worms.

Name	Capacity	Run in alert mode
☐ ThreatSignaturesBotnetWebActionOrder Signatures that detect http botnets.	3500	☐ Disabled
☐ ThreatSignaturesBotnetActionOrder Signatures that are autogenerated from several sources of known and confirmed active botnet and other Command and Control (C2) hosts.	4200	☐ Disabled
☒ ThreatSignaturesMalwareActionOrder Signatures that detect malware (tcp, udp, smtp, icmp, smb, ip) and worm.	1300	☐ Disabled
☐ ThreatSignaturesEmergingEventsActionOrder Signatures with rules developed in response to active and short-lived campaigns and other temporary, high-profile events.	200	☐ Disabled
☐ ThreatSignaturesSuspectActionOrder Signatures to fingerprint malicious SSL certificates using JA3 hashes, identify traffic related to chat clients, and detect suspicious and anomalous user agents.	300	☐ Disabled

Figure 8 - AWS managed threat signature rule group used to detect malware-related network activity.

5.5 Route 53 Resolver DNS Firewall

The Network Firewall domain list restricts HTTP and HTTPS access, but attackers may still abuse DNS to resolve suspicious domains or tunnel data. Route 53 Resolver DNS Firewall closes this gap by blocking DNS queries before resolution occurs.

DNS Firewall rule group: Spoke-VPC-DNS-Firewall

Rule 1: Blocklist

Domain list: RestrictedDomains

Action: BLOCK

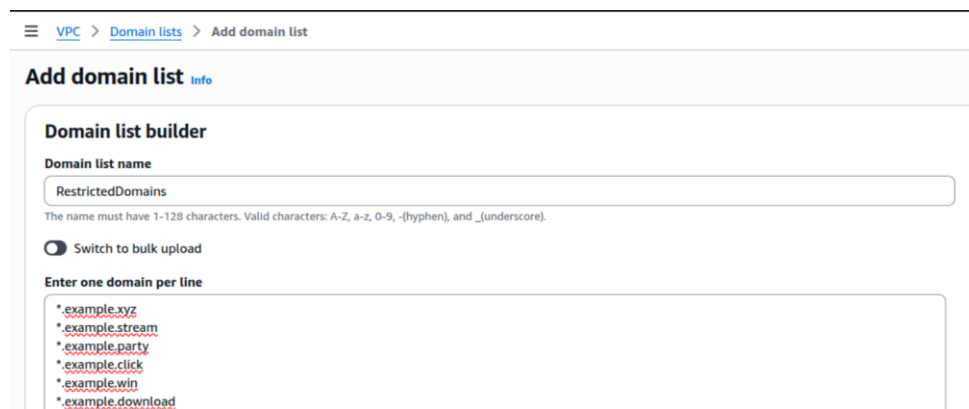
Response: NXDOMAIN

Rule 2: ManagedBlocklist

Domain list: AWSManagedDomainsAggregateThreatList

Action: BLOCK

Response: NXDOMAIN



The screenshot shows the 'Add domain list' page in the AWS Route 53 Resolver console. The 'Domain list name' field is populated with 'RestrictedDomains'. Below this, there is a section for 'Enter one domain per line' with a text area containing several example domains: *.example.xyz, *.example.stream, *.example.party, *.example.click, *.example.win, and *.example.download. The page also includes a 'Switch to bulk upload' toggle and a note about the naming conventions for domain lists.

Figure 9 - DNS Firewall custom domain list used to block suspicious domains associated with simulated attacker infrastructure.

Configuration

Rule type
Defines a filtering rule for DNS queries in a DNS Firewall rule group.

☒ **Domain lists**
Choose your own domain list or an AWS managed domain list

☐ **DNS Firewall Advanced protections**
Select from a range of managed protections and decide the confidence threshold. [View pricing.](#)

Domain lists

Domain lists
You can choose your own domain list or an AWS managed domain list. You can't change the domain list of a rule after you create the rule. [Learn more.](#)

☐ **Custom domain list**
Domain lists you create incur an additional cost. [View pricing.](#)

☒ **AWS managed domain list**
These are subscribed domain lists provided by Amazon.

Choose a domain list

Aggregate Threat List

Domain redirection setting
You can choose to inspect all the domains or only the first domain in the DNS redirection chain, such as CNAME, DNAME, and ALIAS. See [Amazon Route 53 DNS Firewall rule settings](#) for valid domain redirection setting types.

☒ **Inspect all (Default)**
Inspects all the domains in the DNS redirection chain. You need to add individual domains in the redirection chain to the domain list.

☐ **Trust redirection domains**
Inspects only the first domain in the DNS redirection chain. You don't need to add the individual domains in the redirection chain to the domain list.

Figure 10 - DNS Firewall managed threat domain list configured to block known malicious domains.

[Associate with an AWS Firewall Manager policy](#)

Rules | **Associated VPCs** | **Tags**

Associated VPCs (2)

[View details](#) [Disassociate](#) [Associate VPC](#) [Manage tags](#)

	ID	Name	Association ID	Status	Associated priority
☐	vpc-003ab96ddf5dacb24	rgassoc-vpc-003ab96ddf5dacb24-rslvr-frg-af6ac4e276644c	rslvr-frgassoc-faa1ab23358d4ba2	Updating	101
☐	vpc-081451f8dc0563f3e	rgassoc-vpc-081451f8dc0563f3e-rslvr-frg-af6ac4e276644c	rslvr-frgassoc-e128c7154cbd49c6	Updating	101

Figure 11 - DNS Firewall rule group associated with the spoke VPCs to inspect DNS queries from EC2 workloads.

5.6 CloudWatch Logging and Telemetry

Detection requires telemetry. AWS Network Firewall alert logs and Route 53 Resolver DNS query logs were forwarded to CloudWatch. This provides the data required to hunt for blocked domains, protocol anomalies, ICMP patterns, and DNS exfiltration attempts.

Telemetry sources:

- /Lab/NetworkFirewall/Alert
- /Lab/Route53/QueryLogs

Analytics tools:

- CloudWatch Log Insights
- CloudWatch Contributor Insights

Configure query logging

Query logging configuration name
 Name: Spoke-VPC-Query-Logs
 The name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, space, _ (underscore) and - (hyphen)

Query logs destination
 Resolver can save logs in CloudWatch Logs, in an S3 bucket, or in Kinesis Data Streams.
 Destination for query logs: ☒ CloudWatch Logs log group (You can analyze logs with Log Insights and create metrics and alarms.) ☐ S3 bucket (An S3 bucket is economical for long-term log archiving. Latency is typically higher.) ☐ Amazon Data Firehose delivery stream (You can stream logs in real time to Elasticsearch, Redshift, or other applications.)

CloudWatch Logs log groups
 You can either choose a CloudWatch Logs log group that was created by the current account, or choose to create a log group for this query logging configuration.
 /Lab/Route53/QueryLogs

VPCs to log queries for - optional (2/2)
 Resolver logs DNS queries that originate in the VPCs that you choose here. If you don't choose any VPCs, Resolver doesn't log any queries.

VPC ID	VPC name	Status	IPv4 CIDR	IPv6 CIDR	Owner
☒ vpc-003ab96ddf5dabc24	Spoke-VPC-B	Active	10.2.0.0/16	-	152364998533
☒ vpc-081451f8dc0563f3e	Spoke-VPC-A	Active	10.1.0.0/16	-	152364998533

Figure 12 - Route 53 Resolver query logging configuration sending DNS query logs from spoke VPCs to CloudWatch Logs.

6. Threat Hunting Methodology

The hunting workflow starts with broad log visibility and progressively narrows to specific indicators. The objective is not only to detect an event, but to attribute it to a workload and decide whether containment is required.

Step	Question	Telemetry
1	Are firewall rules generating alerts?	Network Firewall alert logs
2	Which hosts contacted blocked domains?	TLS SNI and source IP fields
3	Which hosts generate excessive ICMP?	Firewall protocol field
4	Which hosts use port 443 without TLS?	Suricata alert signature
5	Which instance sends blocked DNS bursts?	DNS query logs + Contributor Insights

7. Detection Engineering and CloudWatch Queries

7.1 Baseline Firewall Alerts

The first query establishes whether firewall alerts are present and provides timestamps for further investigation.

```
fields @timestamp, @message
| display event.timestamp
```

7.2 Blocked Domain Access

This query identifies EC2 sources attempting to reach domains that matched the denylisted FQDN rule.

```
fields @timestamp, @message
| filter event.alert.signature like /denylisted FQDNs/
| display event.tls.sni, event.src_ip
```



Figure 13 - CloudWatch Log Insights results showing blocked domain access attempts from an EC2 instance.

7.3 ICMP Reconnaissance Detection

Repeated ICMP events from a single source suggest reconnaissance or host discovery. This query surfaces the source IPs generating ICMP traffic.

```
fields @timestamp, @message
| filter event.proto like /ICMP/
| display event.src_ip
| sort event.src_ip desc
```

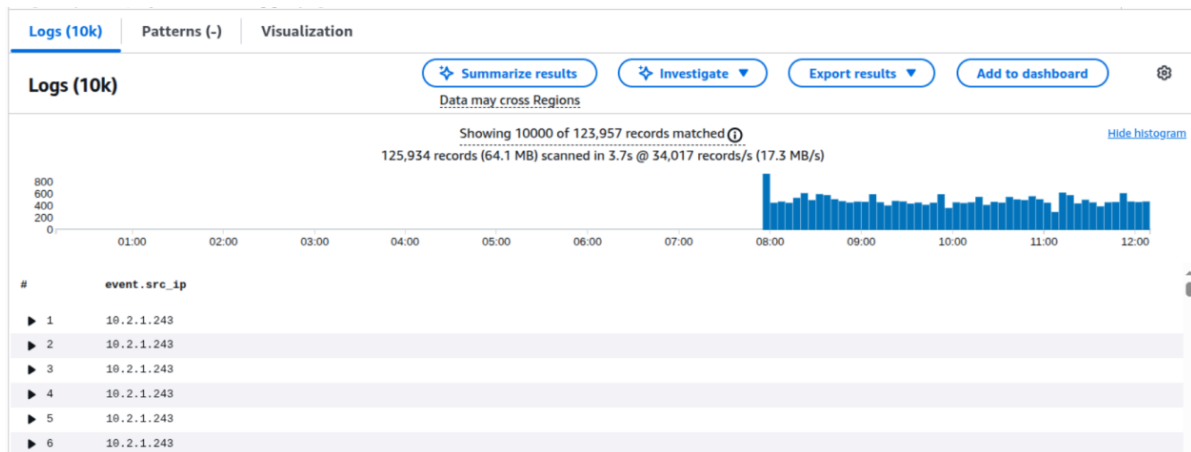


Figure 14 - CloudWatch detection results identifying ICMP reconnaissance activity from a potentially compromised EC2 instance.

7.4 Protocol Abuse Detection

This query isolates Suricata-generated alerts. In this scenario, it identifies non-TLS traffic over TCP/443, consistent with protocol tunneling or suspicious misuse of a commonly allowed port.

```
fields @timestamp, @message
| filter event.alert.signature like /SURICATA/
| display event.src_ip
| sort event.src_ip desc
```

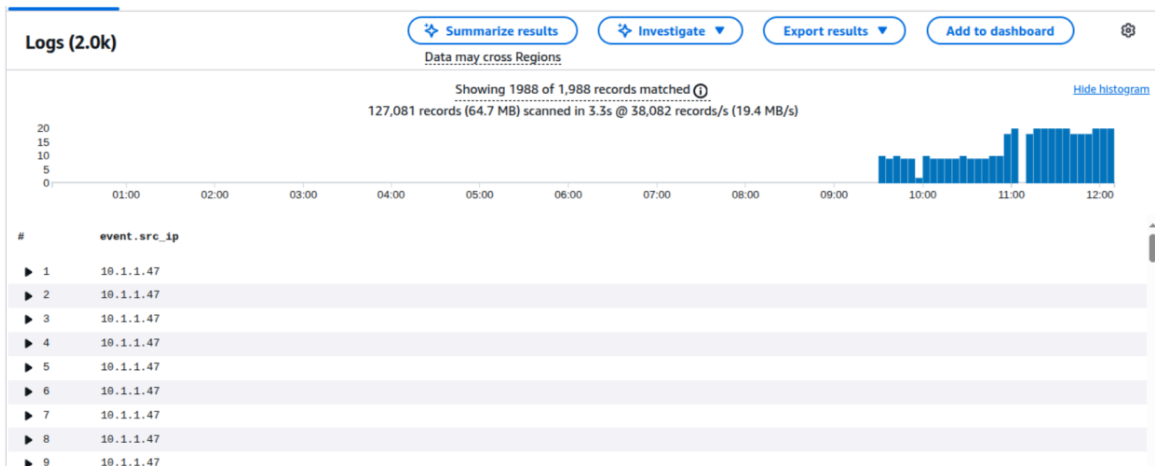


Figure 15 - CloudWatch Log Insights results showing Suricata alerts for non-TLS traffic over TCP port 443.

```

@entity.Attributes.AWS.Resource.ARN arn:aws:network-firewall:us-west-2:152364998533:firewall/InspectionFirewall
@ingestionTime 1761387289744
@log 152364998533:/Lab/NetworkFirewall/Alert
@logStream /aws/network-firewall/alert/InspectionFirewall_2025-10-25-10
@message
{
  "firewall_name": "InspectionFirewall",
  "availability_zone": "us-west-2a",
  "event_timestamp": "1761387121",
  "event": {
    "app_proto": "ftp",
    "src_ip": "10.1.1.47",
    "src_port": "55248",
    "event_type": "alert",
    "alert": {
      "severity": 3,
      "signature_id": "2271003",
      "rev": 1,
      "signature": "SURICATA Port 443 but not TLS",
      "action": "allowed",
      "category": "",
      "flow_id": "402138721398657",
      "dest_ip": "16.145.179.142",
      "proto": "TCP",
      "verdict": {
        "action": "alert",
        "app_proto_ts": "failed",
        "dest_port": "443",
        "pkt_src": "geneve encapsulation",
        "timestamp": "2025-10-25T10:12:01.951730+0000",
        "direction": "to_server"
      }
    }
  }
}
@timestamp 1761387121000
availability_zone us-west-2a
event.alert.action allowed
event.alert.rev 1
event.alert.severity 3
event.alert.signature SURICATA Port 443 but not TLS
event.alert.signature_id 2271003
event.app_proto ftp
event.app_proto_ts failed
event.dest_ip 16.145.179.142
event.dest_port 443
event.direction to_server

```

Figure 16 - AWS Network Firewall alert log containing Suricata metadata such as protocol, source IP, destination IP, and signature.

8. DNS Exfiltration Analysis

The strongest finding came from DNS query telemetry. Contributor Insights showed many blocked DNS queries from the same instance to the same domain. The queried subdomains were long and Base64-like, which is a common indicator of data embedded into DNS labels.

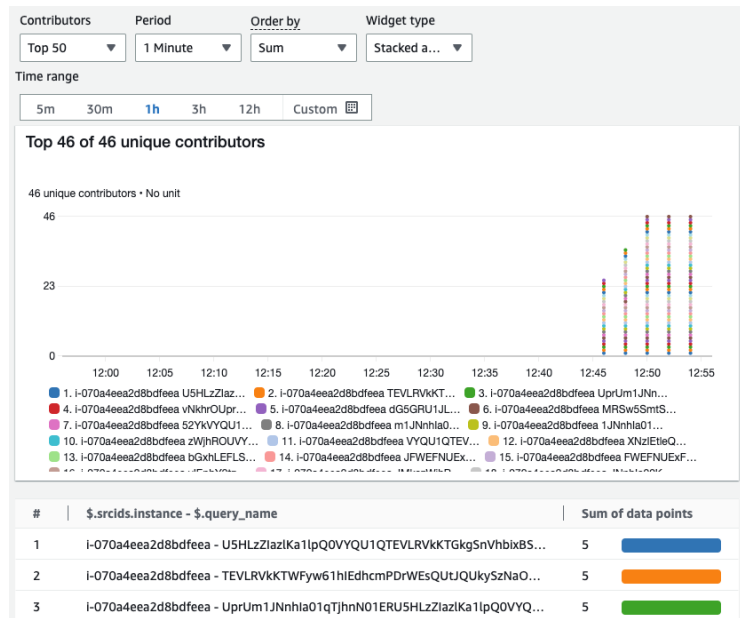


Figure 17 - Contributor Insights view showing blocked DNS queries with encoded-looking subdomains linked to a single EC2 instance.

8.1 Exfiltration Pattern

Compromised EC2 instance
 -> data encoded into Base64
 -> encoded value placed in DNS subdomain
 -> query sent to attacker-controlled domain
 -> DNS Firewall blocks query and logs source instance

8.2 Decoding the Payload

The first DNS label was extracted and decoded from Base64. The decoded output showed that the attacker attempted to exfiltrate AWS credential-like material.

```
awk -F "." '{print $1}' dnsqueries.txt | base64 --decode
```

Decoded payload summary:
 Name, Access Key, Secret Access Key
 <sample rows containing example AWS access key material>

In a production incident, this finding would require immediate credential revocation, IAM access key rotation, CloudTrail review, and investigation of how the instance obtained or stored the credential material.

9. Incident Response Actions

The investigation identified three suspicious EC2 instances associated with the simulated activity. Instead of terminating them immediately, the response action was to quarantine the instances. Quarantine reduces the risk of further lateral movement or exfiltration while preserving the workload for investigation.

Finding	Likely source	Response action
ICMP reconnaissance	Spoke-VPC-TestInstance1 / source IP observed in logs	Apply QuarantineSG and investigate

Non-TLS traffic on TCP/443	Spoke-VPC-TestInstance2 / source IP observed in Suricata alerts	Apply QuarantineSG and inspect process/network activity
DNS exfiltration	Spoke-VPC-TestInstance3 / instance ID from DNS logs	Apply QuarantineSG, revoke credentials, preserve evidence

Containment workflow:

```
EC2 Console -> Instances -> Actions -> Security -> Change security groups
Remove existing security group
Attach QuarantineSG
Save changes
```

A practical QuarantineSG should contain no inbound rules and no outbound internet access. If forensic access is required, outbound connectivity can be limited to the required VPC endpoints for AWS Systems Manager Session Manager.

10. Security Outcomes

- All internet-bound traffic from spoke VPCs is inspected through a central Network Firewall path.
- Suspicious HTTP/HTTPS domain access is denied by stateful firewall rules.
- Protocol misuse on TCP/443 is detected using Suricata-compatible logic.
- DNS Firewall blocks suspicious resolution attempts and records DNS telemetry.
- CloudWatch Log Insights and Contributor Insights identify suspicious EC2 sources.
- Potentially compromised instances are contained using a quarantine security group.

11. Lessons Learned

Centralized egress inspection improves visibility: A hub-and-spoke inspection pattern provides a single enforcement point for outbound traffic from multiple VPCs.

DNS telemetry is critical: DNS can be abused for command-and-control and data exfiltration. Blocking and logging DNS queries materially improves detection capability.

Port-based assumptions are weak: Allowing TCP/443 does not mean the traffic is HTTPS. Application-layer protocol inspection is required to detect port/protocol mismatches.

Logs need hunting logic: Raw logs do not equal detection. CloudWatch queries and Contributor Insights rules turn telemetry into actionable findings.

Containment should preserve evidence: Quarantining instances is often safer than immediate termination because it limits damage while preserving evidence for analysis.